Information–Theoretic Barriers for Collatz Proofs

Craig Alan Feinstein

Contents

Information–Theoretic Barriers for Collatz Proofs

Craig A. Feinstein

Machine-Checked Formalization in Isabelle/HOL

Abstract

In an earlier paper, *The Collatz* $3n + 1$ *Conjecture is Unprovable* (2012), the author argued that any proof of the Collatz conjecture would need to contain arbitrarily large amounts of information about the parity pattern of Collatz trajectories, making a finite proof impossible. The present work formalises that idea in Isabelle/HOL. Under explicit assumptions about how proofs store and preserve information, we prove that no finite proof can establish the required convergence property. This yields a machine-checked information-theoretic barrier theorem inspired by the earlier argument.

Provenance

The present development formalises the information–theoretic ideas underlying the author's earlier paper:

> C. A. Feinstein, *"The Collatz* $3n + 1$ *Conjecture is Unprovable"*, arXiv:math/0312309; *Global Journal of Science Frontier Research*, Mathematics and Decision Sciences, Volume 12, Issue 8 (2012), 13–15.

The assumptions required for that argument are made explicit and formalised within Isabelle/HOL, yielding a machine-checked theorem showing that no finite proof can exist within the corresponding class of proof systems. These assumptions are motivated by structural properties of the Collatz map, including the realisability of arbitrary parity vectors, the injective dependence of affine parameters on parity traces, and the essential role of parity information in determining the dynamics.

The author of this formalisation received assistance from two AI systems — ChatGPT (OpenAI) and Claude (Anthropic). Their assistance consisted of drafting and refining explanatory text, improving the readability of the introduction and comments, and helping diagnose or structure Isabelle/HOL proof scripts.

Main goal

The present development establishes an information-theoretic barrier for proof methods that explicitly store the required parity information.

High-level strategy

The argument formalised here isolates the following phenomena:

1. If a proposed proof has length L, an incompressible parity vector of length $L + 1$ can be chosen.
2. The Collatz map realises that vector as the first $L + 1$ parities of a positive trajectory, while also making the parities at steps L and $L+1$ equal.
3. The equality of these two parities implies that the value at step L is greater than 2, and therefore any step at which the trajectory equals 1 must occur after step L.
4. Under the trace–specification assumption, the proposed proof must contain an encoding of the chosen $L + 1$ bits, contradicting its length L.

The structure of the argument is inspired by Chaitin–style incompressibility methods, but is applied to the representation of computational traces within proof certificates rather than to the computation of specific strings.

Structure of the formalisation

1. **Collatz map and parity vectors** We define the Collatz function T and formalise parity vectors as computational traces of the iterative process.

2. **Affine formula characterisation** We show that $T^{(k)}(n)$ admits an affine representation $(3^s \cdot n + c)/2^k$, and prove that the parameters (k, s, c) are uniquely determined by the parity vector.

3. **Two–adic invariance** We establish a key invariance property: adding 2^k to a starting value does not affect the first k parity bits. This property underlies the realisability construction.

4. **Every parity vector is realisable** We prove that every finite binary string occurs as the parity vector of some starting value. This shows that arbitrarily complex computational traces are inherent to the Collatz dynamics.

5. **Proof system setup** We introduce an abstract model of proof certificates based on bitstrings, substring containment, and information–theoretic compressibility.

6. **The original information–barrier argument** We formalise the original information–theoretic argument for the Collatz conjecture, with its proof–system requirements stated explicitly.

```
theory Collatz_Information_Barriers
  imports Main
begin
```

1 Collatz map and parity vectors

The Collatz function

We define the Collatz function T by

$$T(n) = \begin{cases} n/2 & \text{if } n \text{ is even,} \\ (3n+1)/2 & \text{if } n \text{ is odd.} \end{cases}$$

```
definition T :: "nat ⇒ nat" where
  "T n = (if even n then n div 2 else (3*n + 1) div 2)"
```

```
abbreviation Tpow :: "nat ⇒ nat ⇒ nat"
  where "Tpow k n ≡ (T ^^ k) n"
```

The Collatz conjecture

The formulation of the conjecture used in the earlier paper, and throughout this development, is:

$$\forall n > 0.\ \exists k.\ T^{(k)}(n) = 1.$$

The parity vector as computational trace

The parity vector *parity_vec n k* records the sequence of even and odd values encountered during the first k iterations starting from n:

$$\textit{parity\_vec}\ n\ k = [\ \textit{odd}(n),\ \textit{odd}(T(n)),\ \textit{odd}(T^{(2)}(n)),\ \ldots,\ \textit{odd}(T^{(k-1)}(n))\ ].$$

This parity vector represents the *computational trace*: the sequence of branching decisions made while repeatedly applying the Collatz map.

Example

For $n = 27$ we have:

$$T(27) = 41, \quad T^{(2)}(27) = 62, \quad T^{(3)}(27) = 31.$$

Thus:

$$\begin{aligned}\textit{parity\_vec}\ 27\ 4 &= [\ \textit{odd}(T^{(0)}(27)),\ \textit{odd}(T^{(1)}(27)),\ \textit{odd}(T^{(2)}(27)),\ \textit{odd}(T^{(3)}(27))\ ] \\ &= [\ \textit{odd}(27),\ \textit{odd}(41),\ \textit{odd}(62),\ \textit{odd}(31)\ ] \\ &= [\ \text{True},\ \text{True},\ \text{False},\ \text{True}\ ].\end{aligned}$$

```
definition collatz_conjecture :: bool where
  "collatz_conjecture ⟷ (∀n>0. ∃k. Tpow k n = 1)"

definition parity_vec :: "nat ⇒ nat ⇒ bool list"
  where "parity_vec n k = map (λi. odd (Tpow i n)) [0..<k]"

lemma length_parity_vec[simp]: "length (parity_vec n k) = k"
  by (simp add: parity_vec_def)
```

2 Affine formula characterisation

After k iterations of the Collatz map T, the result admits an affine representation:

$$T^{(k)}(n) = \frac{3^s \cdot n + c}{2^k}.$$

Here:

- k is the number of iterations.
- s counts the number of odd values in the parity vector, that is, the number of steps of the form $(3n+1)/2$.
- c is a constant determined by the specific parity sequence.

Intuition

Each odd step multiplies the current value by 3, adds 1, and then divides by 2, while each even step simply divides by 2. After k steps, the cumulative effect is multiplication by 3^s, division by 2^k, and the addition of an accumulated constant c.

Uniqueness

The parameters (k, s, c) are uniquely determined by the parity vector. In particular, the parity vector encodes all computational information needed to reconstruct the affine formula for $T^{(k)}(n)$.

```
fun params :: "nat ⇒ bool list ⇒ nat × nat" where
  "params i [] = (0, 0)" |
  "params i (b # bs) =
     (let (c,s) = params (Suc i) bs in
      if b then (3*c + 2^i, Suc s) else (c, s))"

definition params0 :: "bool list ⇒ nat × nat" where
  "params0 x = params 0 x"

definition formula_of :: "bool list ⇒ nat × nat × nat" where
  "formula_of x = (length x, snd (params0 x), fst (params0 x))"
```

Injectivity

Different parity vectors produce distinct triples (k, s, c) in the representation

$$T^{(k)}(n) = \frac{3^s \cdot n + c}{2^k}.$$

```
lemma pow2_dvd_fst_params_Suc:
  "2 ^ Suc i dvd fst (params (Suc i) zs)"
proof (induction zs arbitrary: i)
  case Nil
  show ?case by simp
next
  case (Cons b bs)
  obtain c s where P: "params (Suc (Suc i)) bs = (c, s)"
    by (cases "params (Suc (Suc i)) bs") auto
```

```
  from Cons.IH[of "Suc i"] P have IH: "2 ^ Suc (Suc i) dvd c" by simp
  then obtain t where c_rep: "c = 2 ^ Suc (Suc i) * t"
    by (auto simp: dvd_def)
  have div_c: "2 ^ Suc i dvd c"
  proof (unfold dvd_def, intro exI)
    show "c = 2 ^ Suc i * (2 * t)"
      by (simp add: c_rep)
  qed
  with P show ?case
    by (cases b) simp_all
qed

lemma mod_drop_left_multiple_nat:
  fixes m a r :: nat
  shows "(m * a + r) mod m = r mod m"
  by (simp add: mod_add_left_eq mod_mult_left_eq)

lemma params_injective_len:
  assumes "length xs = length ys" and "params i xs = params i ys"
  shows   "xs = ys"
  using assms
proof (induction xs arbitrary: ys i)
  case Nil
  then show ?case by (cases ys) auto
next
  case (Cons a xs)
  from Cons.prems(1) obtain b ys' where [simp]: "ys = b # ys'"
    by (cases ys) auto

  obtain c s where Pxs: "params (Suc i) xs = (c,s)"
    by (cases "params (Suc i) xs") auto
  obtain c' s' where Pys: "params (Suc i) ys' = (c',s')"
    by (cases "params (Suc i) ys'") auto

  have Eq:
    "(if a then (3*c + 2^i, Suc s) else (c,s)) =
     (if b then (3*c' + 2^i, Suc s') else (c',s'))"
    using Cons.prems(2) Pxs Pys by simp

  have len_tails: "length xs = length ys'" using Cons.prems(1) by simp

  show ?case
  proof -
    have "a = b"
    proof (rule ccontr)
      assume "a ≠ b"
      then consider (TF) "a" "¬ b" | (FT) "¬ a" "b" by auto
      then show False
      proof cases
```

```
  case TF
  from pow2_dvd_fst_params_Suc[of i xs] Pxs
  have div_c:  "2 ^ Suc i dvd c"  by simp
  from pow2_dvd_fst_params_Suc[of i ys'] Pys
  have div_c': "2 ^ Suc i dvd c'" by simp
  let ?M = "2 ^ Suc i"

  obtain t  where c_rep:  "c  = ?M * t"
    using div_c  by (auto simp: dvd_def)
  obtain t' where c'rep: "c' = ?M * t'"
    using div_c' by (auto simp: dvd_def)

  have Lmod: "(3 * c + 2 ^ i) mod ?M = 2 ^ i"
  proof -
    have "(3 * c + 2 ^ i) mod ?M
        = ((?M * (3 * t)) + 2 ^ i) mod ?M"
      by (simp add: c_rep algebra_simps)
    also have "... = (2 ^ i) mod ?M"
      by (meson mod_drop_left_multiple_nat)
    also have "... = 2 ^ i" by simp
    finally show ?thesis .
  qed

  have Rmod: "c' mod ?M = 0" by (simp add: c'rep)

  from Eq TF have "(3*c + 2^i, Suc s) = (c', s')" by simp
  hence "(3*c + 2^i) = c'" by simp
  hence "(3*c + 2^i) mod ?M = c' mod ?M" by simp
  hence "2 ^ i = 0" using Lmod Rmod by simp
  thus False using power_eq_0_iff by fastforce
next
  case FT
  from pow2_dvd_fst_params_Suc[of i ys'] Pys
  have div_c': "2 ^ Suc i dvd c'" by simp
  from pow2_dvd_fst_params_Suc[of i xs]  Pxs
  have div_c:  "2 ^ Suc i dvd c"  by simp
  let ?M = "2 ^ Suc i"

  obtain t' where c'rep: "c' = ?M * t'"
    using div_c' by (auto simp: dvd_def)
  obtain t  where c_rep:  "c  = ?M * t"
    using div_c  by (auto simp: dvd_def)

  have Rmod: "(3 * c' + 2 ^ i) mod ?M = 2 ^ i"
  proof -
    have "(3 * c' + 2 ^ i) mod ?M
        = ((?M * (3 * t')) + 2 ^ i) mod ?M"
      by (simp add: c'rep algebra_simps)
    also have "... = (2 ^ i) mod ?M"
```

```
        using mod_drop_left_multiple_nat by blast
      also have "... = 2 ^ i" by simp
      finally show ?thesis .
    qed

    have Lmod: "c mod ?M = 0" by (simp add: c_rep)

    from Eq FT have "(c, s) = (3*c' + 2^i, Suc s')" by simp
    hence "c = 3*c' + 2^i" by simp
    hence "c mod ?M = (3*c' + 2^i) mod ?M" by simp
    hence "0 = 2 ^ i" using Lmod Rmod by simp
    thus False by (metis not_exp_less_eq_0_int verit_comp_simplify1(2))
  qed
 qed

 then have bits_eq: "a = b" by simp
 from Eq bits_eq have "c = c' ∧ s = s'" by (cases a) auto
 hence "xs = ys'"
   using Cons.IH[OF len_tails] Pxs Pys by metis
 with bits_eq show ?thesis by simp
 qed
qed


lemma formula_determines_parity_on_len:
 assumes "length x = k" "length y = k" "formula_of x = formula_of y"
 shows   "x = y"
proof -
 from assms(3)
 have "snd (params0 x) = snd (params0 y)"
      "fst (params0 x) = fst (params0 y)"
   by (auto simp: formula_of_def)
 hence "params0 x = params0 y"
   by (cases "params0 x"; cases "params0 y"; simp)
 thus ?thesis
   using assms(1,2) params_injective_len[of x y 0]
   by (simp add: params0_def)
qed
```

3 Two-adic invariance

Key lemma

Adding $q \cdot 2^k$ to m preserves the first k parity bits:

$$parity\_vec(m + q \cdot 2^k, k) = parity\_vec(m, k).$$

Intuition

The offset $q \cdot 2^k$ is beyond the resolution of the first k steps. After one application of the Collatz map T, the offset becomes $q \cdot 2^{k-1}$ if m is even, or $3q \cdot 2^{k-1}$ if m is odd, and therefore remains a multiple of 2^{k-1}. This behaviour continues inductively through subsequent iterations.

Consequence

This invariance property enables realisability. One can tune a number to have any desired parity sequence by adding suitable multiples of 2^k, without affecting parity bits that have already been fixed.

```
lemma T_funpow_commute: "T ((T ^^ j) n) = (T ^^ j) (T n)"
proof (induction j)
  case 0
  show ?case by simp
next
  case (Suc j)
  have "T ((T ^^ Suc j) n) = T (T ((T ^^ j) n))" by simp
  also have "... = T ((T ^^ j) (T n))" by (simp add: Suc.IH)
  also have "... = (T ^^ Suc j) (T n)" by simp
  finally show ?case .
qed

lemma parity_vec_Suc:
  "parity_vec n (Suc k) = odd n # parity_vec (T n) k"
proof (rule nth_equalityI)
  show "length (parity_vec n (Suc k)) =
        length (odd n # parity_vec (T n) k)"
    by simp
next
  fix i assume iLt: "i < length (parity_vec n (Suc k))"
  then have iSk: "i < Suc k" by (simp add: parity_vec_def)
  consider (Z) "i = 0" | (S) j where "i = Suc j" "j < k"
    using iSk by (cases i) auto
  then show  "parity_vec n (Suc k) ! i =
              (odd n # parity_vec (T n) k) ! i"
  proof cases
    case Z
    show ?thesis
      using Z by (simp add: parity_vec_def del: upt_Suc)
  next
    case (S j)
    show ?thesis
      using S
      by (simp add: parity_vec_def T_funpow_commute del: upt_Suc)
  qed
qed
```

```
lemma parity_vec_add_pow2_invariant:
  fixes m q k :: nat
  shows "parity_vec (m + q * 2 ^ k) k = parity_vec m k"
proof (induction k arbitrary: m q)
  case 0
  show ?case by (simp add: parity_vec_def)
next
  case (Suc k)
  let ?Δ = "q * 2 ^ Suc k"

  have head: "odd (m + ?Δ) = odd m"
    by simp

  have tail: "parity_vec (T (m + ?Δ)) k = parity_vec (T m) k"
  proof (cases "even m")
    case True
    then obtain t where m2: "m = 2*t" by (elim evenE)
    have "(m + ?Δ) div 2 = t + q * 2 ^ k"
      by (simp add: m2)
    hence "T (m + ?Δ) = t + q * 2 ^ k"
      using True by (simp add: T_def)
    moreover have "T m = t"
      using True m2 by (simp add: T_def)
    ultimately show ?thesis
      using Suc.IH[of t q] by simp
  next
    case False
    then obtain t where m2: "m = 2*t + 1" by (elim oddE)
    have "(3*(m + ?Δ) + 1) div 2
          = (3*m + 1) div 2 + (3*q) * 2 ^ k"
    proof -
      have "3*(m + ?Δ) + 1 = (3*m + 1) + (3*q) * 2 ^ Suc k"
        by simp
      moreover have "even (3*m + 1)"
        using False m2 by simp
      ultimately show ?thesis by simp
    qed
    hence "T (m + ?Δ) = T m + (3*q) * 2 ^ k"
      using False by (simp add: T_def)
    thus ?thesis
      using Suc.IH[of "T m" "3*q"] by simp
  qed

  have step1: "parity_vec (m + ?Δ) (Suc k) =
               odd (m + ?Δ) # parity_vec (T (m + ?Δ)) k"
    by (simp add: parity_vec_Suc)
  also have step2: "... = odd m # parity_vec (T m) k"
    using head tail by blast
```

```
  also have step3: "... = parity_vec m (Suc k)"
    by (simp add: parity_vec_Suc)
  finally show ?case .
qed
```

4 Every parity vector is realisable

Realisability

For any finite binary string x, there exists a natural number n whose parity sequence agrees with x:

$$\forall x.\ \exists n.\ \mathit{parity\_vec}\ n\ (\text{length}\ x) = x.$$

This means that every possible computational trace of the Collatz map actually occurs for some starting number. In particular, parity traces of arbitrary length, including incompressible traces, are realised.

Helper lemmas for modular arithmetic

The following lemmas establish basic modular-arithmetic facts that are used in the realisability construction.

```
lemma power_mod_nat:
  fixes a m n :: nat
  shows "(a ^ n) mod m = ((a mod m) ^ n) mod m"
proof (induction n)
  case 0
  show ?case by simp
next
  case (Suc n)
  have "(a ^ Suc n) mod m = (a * a ^ n) mod m" by simp
  also have "... = (((a mod m) * a ^ n) mod m)"
    by (simp add: mod_mult_left_eq)
  also have "... = (((a mod m) * ((a ^ n) mod m)) mod m)"
    by (simp add: mod_mult_right_eq)
  also have "... = (((a mod m) * (((a mod m) ^ n) mod m)) mod m)"
    by (simp add: Suc.IH)
  also have "... = (((a mod m) * ((a mod m) ^ n)) mod m)"
    by (simp add: mod_mult_right_eq)
  also have "... = ((a mod m) ^ Suc n) mod m" by simp
  finally show ?case .
qed


lemma power_mult_nat:
  fixes a :: nat
  shows "a ^ (m * n) = (a ^ m) ^ n"
```

```
proof (induction n)
  case 0
  show ?case by simp
next
  case (Suc n)
  have "a ^ (m * Suc n) = a ^ (m*n + m)" by (simp add: add.commute)
  also have "... = a ^ (m*n) * a ^ m" by (simp add: power_add)
  also have "... = (a ^ m) ^ n * a ^ m" by (simp add: Suc.IH)
  also have "... = (a ^ m) ^ Suc n" by simp
  finally show ?case .
qed

lemma pow4_mod3: "((4::nat) ^ m) mod 3 = 1"
  by (simp add: power_mod_nat)

lemma pow2_mod3_even:
  assumes "even l"
  shows "(2 :: nat) ^ l mod 3 = 1"
proof -
  obtain m where L: "l = 2*m" using assms by (erule evenE)
  have "2 ^ l mod 3 = (2 ^ (2*m)) mod 3" by (simp add: L)
  also have "... = (((2::nat) ^ 2) ^ m) mod 3" by (simp add: power_mult_nat)
  also have "... = (4 ^ m) mod 3" by simp
  also have "... = 1" by (rule pow4_mod3)
  finally show ?thesis .
qed

lemma pow2_mod3_odd:
  assumes "odd l"
  shows "(2 :: nat) ^ l mod 3 = 2"
proof -
  obtain m where L: "l = Suc (2*m)" using assms
    by (metis Suc_eq_plus1 oddE)
  have "2 ^ l mod 3 = (2 * 2 ^ (2*m)) mod 3" by (simp add: L)
  also have "... = (2 * (((2::nat) ^ 2) ^ m)) mod 3"
    by (simp add: power_mult_nat)
  also have "... = (2 * ((4 ^ m) mod 3)) mod 3"
    by (simp add: mod_mult_right_eq)
  also have "... = (2 * 1) mod 3" by (simp add: pow4_mod3)
  also have "... = 2" by simp
  finally show ?thesis .
qed

lemma pow2_mod3:
  fixes l :: nat
  shows "(2 :: nat) ^ l mod 3 = (if even l then 1 else 2)"
  by (cases "even l") (simp add: pow2_mod3_even, simp add: pow2_mod3_odd)

lemma choose_t_even_mod3:
```

```
  fixes m0 l :: nat
  assumes "even l"
  shows "∃t≤2. (m0 + t * (2 :: nat) ^ l) mod 3 = 2"
proof -
  define t where "t = (2 - (m0 mod 3)) mod 3"
  have t_le2: "t ≤ 2" by (simp add: t_def)
  have "(m0 + t * 2 ^ l) mod 3
          = (m0 mod 3 + t * (2 ^ l mod 3)) mod 3"
    by (metis mod_add_cong mod_mod_trivial mod_mult_right_eq)
  also have "... = (m0 mod 3 + t) mod 3"
    using assms by (simp add: pow2_mod3)
  also have "... = (m0 mod 3 + ((2 - (m0 mod 3)) mod 3)) mod 3"
    by (simp add: t_def)
  also have "... = 2"
    by (cases "m0 mod 3") simp_all
  finally have targ: "(m0 + t * 2 ^ l) mod 3 = 2" .
  from t_le2 targ show ?thesis by blast
qed

lemma choose_t_odd_mod3:
  fixes m0 l :: nat
  assumes "odd l"
  shows "∃t≤2. (m0 + t * (2 :: nat) ^ l) mod 3 = 2"
proof -
  define t where "t = (2 * (2 - (m0 mod 3))) mod 3"
  have t_le2: "t ≤ 2" by (simp add: t_def)
  have "(m0 + t * 2 ^ l) mod 3
          = (m0 mod 3 + t * (2 ^ l mod 3)) mod 3"
    by (metis mod_add_cong mod_mod_trivial mod_mult_right_eq)
  also have "... = (m0 mod 3 + ((2 * (2 - (m0 mod 3))) mod 3) * 2)
           mod 3"
    by (simp add: t_def pow2_mod3 assms)
  also have "... = (m0 mod 3 + (4 * (2 - (m0 mod 3))) mod 3) mod 3"
    using mod_mult_right_eq by (metis (no_types, lifting)
     distrib_right mod_add_eq mod_add_left_eq mult_2_right numeral_Bit0_eq_double)
  also have "... = (m0 mod 3 + ((4 mod 3) *
    ((2 - (m0 mod 3)) mod 3)) mod 3) mod 3"
    using mod_mult_left_eq by (metis mod_mult_right_eq)
  also have "... = (m0 mod 3 + ((2 - (m0 mod 3)) mod 3)) mod 3" by simp
  also have "... = 2" by (cases "m0 mod 3") simp_all
  finally have targ: "(m0 + t * 2 ^ l) mod 3 = 2" .
  show ?thesis using t_le2 targ by blast
qed

lemma parity_vector_realizable:
  fixes x :: "bool list"
  shows "∃n. parity_vec n (length x) = x"
proof (induction x)
  case Nil
```

```
  show ?case by (metis length_0_conv length_parity_vec)
next
  case (Cons b bs)
  obtain m0 where IH: "parity_vec m0 (length bs) = bs"
    using Cons.IH by blast
  show ?case
  proof (cases b)
    case False
    let ?n = "2*m0"
    have "parity_vec ?n (length (b # bs))
          = odd ?n # parity_vec (T ?n) (length bs)"
      by (simp add: parity_vec_Suc)
    also have "... = False # bs" by (simp add: T_def IH)
    also have "... = b # bs" by (simp add: False)
    finally show ?thesis by (intro exI[of _ ?n])
  next
    case True
    let ?l = "length bs"
    obtain t where t_le2: "t ≤ 2" and
                   targ: "(m0 + t * 2 ^ ?l) mod 3 = 2"
      by (cases "even ?l")
         (use choose_t_even_mod3[of ?l m0]
          choose_t_odd_mod3[of ?l m0] in auto)
    let ?m = "m0 + t * 2 ^ ?l"
    have tail_preserved: "parity_vec ?m ?l = bs"
      using IH parity_vec_add_pow2_invariant[of m0 t ?l] by simp
    define q where "q = ?m div 3"
    have m_eq: "?m = 3*q + 2"
    proof -
      have "?m = 3 * (?m div 3) + (?m mod 3)" by (simp add: div_mult_mod_eq)
      also have "... = 3*q + 2" by (simp add: q_def targ)
      finally show ?thesis .
    qed
    let ?n = "(2 * ?m - 1) div 3"
    have head: "odd ?n"
    proof -
      have "?n = (2 * (3*q + 2) - 1) div 3" by (simp add: m_eq)
      also have "... = (6*q + 3) div 3" by simp
      also have "... = 2*q + 1" by simp
      finally show ?thesis by simp
    qed
    have tail_step: "parity_vec (T ?n) ?l = bs"
      using tail_preserved head by (simp add: m_eq T_def)
    have "parity_vec ?n (length (b # bs)) =
          odd ?n # parity_vec (T ?n) ?l"
      by (simp add: parity_vec_Suc)
    also have "... = True # bs" using head tail_step by simp
    also have "... = b # bs" by (simp add: True)
    finally show ?thesis by (intro exI[of _ ?n])
```

```
  qed
qed
```

Realisability with matching successive parity

Let x be a bit vector of length $L + 1$. There is a positive natural number n such that

$$x = (n, T(n), \ldots, T^{(L)}(n)) \pmod 2$$

and

$$T^{(L+1)}(n) = T^{(L)}(n) \pmod 2.$$

To obtain matching parities at steps L and $L + 1$, append a copy of the final bit of x to the vector. Parity-vector realisability, together with two-adic invariance, then supplies a positive starting value having this extended parity vector.

Since n is positive, every value in its trajectory is positive. If $T^{(L)}(n) = 1$, then $T^{(L+1)}(n) = 2$; if $T^{(L)}(n) = 2$, then $T^{(L+1)}(n) = 1$. In either case, the two values have opposite parities. Their equal parities therefore imply that $T^{(L)}(n) > 2$. If the trajectory had reached 1 at some step $k \leq L$, every subsequent value through step L would belong to the cycle $1, 2, 1, 2, \ldots$, which would give $T^{(L)}(n) \leq 2$. This contradicts $T^{(L)}(n) > 2$. Therefore, $T^{(k)}(n) = 1$ implies $k > L$.

```
lemma parity_vector_realizable_with_matching_next_parity:
  assumes x_len: "length x = Suc L"
  shows "∃n>0.
           parity_vec n (Suc L) = x ∧
           odd (Tpow (Suc L) n) = odd (Tpow L n)"
proof -
  let ?y = "x @ [x ! L]"
  obtain m where m_realizes:
    "parity_vec m (length ?y) = ?y"
    using parity_vector_realizable[of ?y] by blast
  define n where "n = m + 2 ^ Suc (Suc L)"
  have y_len: "length ?y = Suc (Suc L)"
    using x_len by simp
  have m_realizes':
    "parity_vec m (Suc (Suc L)) = ?y"
    using m_realizes y_len by simp
  have invariant:
    "parity_vec n (Suc (Suc L)) =
     parity_vec m (Suc (Suc L))"
    using parity_vec_add_pow2_invariant[of m 1 "Suc (Suc L)"]
    by (simp add: n_def)
  have n_realizes:
    "parity_vec n (Suc (Suc L)) = ?y"
    using invariant m_realizes' by simp
```

```
  have n_pos: "n > 0"
    by (simp add: n_def)
  have prefix:
    "parity_vec n (Suc L) = x"
  proof -
    have "parity_vec n (Suc L) =
          take (Suc L) (parity_vec n (Suc (Suc L)))"
      by (simp add: parity_vec_def take_map)
    also have "... = take (Suc L) ?y"
      by (simp add: n_realizes)
    also have "... = x"
      using x_len by simp
    finally show ?thesis .
  qed
  have at_L:
    "odd (Tpow L n) = x ! L"
  proof -
    have x_nonempty: "x ≠ []"
      using x_len by auto

    have "odd (Tpow L n) =
          last (parity_vec n (Suc L))"
      by (simp add: parity_vec_def)
    also have "... = last x"
      by (simp add: prefix)
    also have "... = x ! (length x - 1)"
      by (rule last_conv_nth[OF x_nonempty])
    also have "... = x ! L"
      using x_len by simp
    finally show ?thesis .
  qed

  have at_Suc_L:
    "odd (Tpow (Suc L) n) = x ! L"
  proof -
    have "odd (Tpow (Suc L) n) =
          last (parity_vec n (Suc (Suc L)))"
      by (simp add: parity_vec_def)
    also have "... = last ?y"
      by (simp add: n_realizes)
    also have "... = x ! L"
      by simp
    finally show ?thesis .
  qed
  show ?thesis
    using n_pos prefix at_L at_Suc_L by blast
qed

lemma T_positive:
```

```
  assumes "n > 0"
  shows "T n > 0"
proof (cases "even n")
  case True
  then obtain q where n_eq: "n = 2 * q"
    by (elim evenE)
  have "q > 0"
    using assms n_eq by simp
  then show ?thesis
    using True by (simp add: T_def n_eq)
next
  case False
  then obtain q where n_eq: "n = 2 * q + 1"
    by (elim oddE)
  then show ?thesis
    by (simp add: T_def)
qed

lemma Tpow_positive:
  assumes "n > 0"
  shows "Tpow k n > 0"
  using assms
proof (induction k arbitrary: n)
  case 0
  then show ?case by simp
next
  case (Suc k)
  have iter_pos: "Tpow k n > 0"
    using Suc.IH Suc.prems by blast
  have "T (Tpow k n) > 0"
    using T_positive[OF iter_pos] .
  then show ?case
    by (simp add: funpow_Suc_right)
qed

lemma equal_successive_parity_imp_gt_two:
  assumes n_pos: "n > 0"
    and same_parity:
      "odd (Tpow (Suc L) n) = odd (Tpow L n)"
  shows "Tpow L n > 2"
proof -
  have step:
    "Tpow (Suc L) n = T (Tpow L n)"
    by (simp add: funpow_Suc_right)
  have iter_pos: "Tpow L n > 0"
    using Tpow_positive n_pos by blast
  have not_one: "Tpow L n ≠ 1"
    using same_parity step by (auto simp: T_def)
  have not_two: "Tpow L n ≠ 2"
```

```
    using same_parity step by (auto simp: T_def)
  show ?thesis
    using iter_pos not_one not_two by linarith
qed

lemma Tpow_one_le_two:
  "Tpow j 1 ≤ 2"
proof -
  have "Tpow j 1 = 1 ∨ Tpow j 1 = 2"
  proof (induction j)
    case 0
    then show ?case by simp
  next
    case (Suc j)
    then show ?case
      by (auto simp: funpow_Suc_right T_def)
  qed
  then show ?thesis by auto
qed

lemma reaches_one_only_after_L:
  assumes n_pos: "n > 0"
    and same_parity:
      "odd (Tpow (Suc L) n) = odd (Tpow L n)"
    and reaches: "Tpow k n = 1"
  shows "k > L"
proof (rule ccontr)
  assume "¬ k > L"
  then have k_le: "k ≤ L" by simp
  have L_decomp: "L = (L - k) + k"
    using k_le by simp
  have "Tpow L n = Tpow ((L - k) + k) n"
    by (rule arg_cong[OF L_decomp])
  also have "... = Tpow (L - k) (Tpow k n)"
    by (simp add: funpow_add)
  also have "... = Tpow (L - k) 1"
    by (simp add: reaches)
  also have "... ≤ 2"
    by (rule Tpow_one_le_two)
  finally have "Tpow L n ≤ 2" .
  moreover have "Tpow L n > 2"
    using equal_successive_parity_imp_gt_two[OF n_pos same_parity] .
  ultimately show False by simp
qed
```

5 Proof system setup

```
type_synonym bit = bool
type_synonym bitstring = "bit list"
```

Substring containment

We introduce a substring-containment relation as a *concrete and explicit* model of proofs that store computational trace information as literal data. Formally, a bitstring p contains a bitstring s if s occurs verbatim as a contiguous substring of p, i.e. if there exist bitstrings u and v such that $p = u@s@v$.

Remark. The choice of substring containment is deliberately strong. It provides a simple, syntactic notion of explicit information storage that is easy to reason about formally and avoids ambiguity about how information is represented inside a proof certificate. In the main barrier theorem, literal containment implies that the encoded parity vector cannot be longer than the proof certificate. The separately established incompressibility condition ensures that the chosen vector cannot be represented by an encoding shorter than the vector itself.

```
definition contains :: "bitstring ⇒ bitstring ⇒ bool"
  where "contains p s ⟷ (∃u v. p = u @ s @ v)"

lemma contains_len_bound: "contains p s ==> length s <= length p"
  by (auto simp: contains_def)
```

6 The unprovability theorem

```
lemma finite_bitstrings_of_len:
  "finite {s::bitstring. length s = m}"
proof -
  have fin_aux:
    "finite {s::bool list. set s <= (UNIV::bool set) ∧ length s = m}"
    by (rule finite_lists_length_eq) simp
  have "{s::bitstring. length s = m}
        = {s. set s <= (UNIV::bool set) ∧ length s = m}"
    by auto
  then show ?thesis using fin_aux by (simp only:)
qed

lemma many_strings_of_length:
  "card {s::bitstring. length s = m} = 2 ^ m"
proof (induction m)
  case 0
  have "{s::bitstring. length s = 0} = {[]}" by auto
  thus ?case by simp
next
  case (Suc m)
  have "{s. length s = Suc m} =
    (%b. True # b) ` {s. length s = m}
    Un (%b. False # b) ` {s. length s = m}"
    by (auto simp: length_Suc_conv)
```

```
  moreover have "(λb. True # b) ` {s. length s = m}
    Int (λb. False # b) ` {s. length s = m} = {}"
    by auto
  moreover have "inj_on (λb. True # b) {s. length s = m}"
    by (auto simp: inj_on_def)
  moreover have "inj_on (λb. False # b) {s. length s = m}"
    by (auto simp: inj_on_def)
  ultimately show ?case
    using Suc.IH card_Un_disjoint card_image
    by (smt (verit) Suc_1 Suc_pred card.infinite diff_add_zero
        mult_2 nat.discI plus_1_eq_Suc power_Suc0_right power_add
        power_eq_0_iff zero_less_one)
qed

definition compressible :: "bitstring ⇒ (bitstring ⇒ bitstring) ⇒ bool"
where
  "compressible s enc ≡ length (enc s) < length s"

definition incompressible_by :: "bitstring ⇒ (bitstring ⇒ bitstring)
⇒ bool" where
  "incompressible_by s enc ≡ ¬compressible s enc"

lemma sum_pow2_lt: "sum (%i. (2::nat) ^ i) {..<m} = 2 ^ m - 1"
  by (induction m) simp_all

lemma finite_bitstrings_le_len:
  "finite {s::bitstring. length s ≤ m}"
proof (induction m)
  case 0 show ?case by (simp add: finite_bitstrings_of_len)
next
  case (Suc m)
  have "{s::bitstring. length s <= Suc m}
      = {s. length s <= m} Un {s. length s = Suc m}" by auto
  thus ?case using Suc.IH finite_bitstrings_of_len by (simp add: finite_UnI)
qed

lemma card_bitstrings_le_len:
  "card {s::bitstring. length s <= m} = sum (%i. 2 ^ i) {..m}"
proof -
  let ?S = "λi. {s::bitstring. length s = i}"
  have union_eq: "{s::bitstring. length s <= m} =
                  Union ((%i. ?S i) ` {..m})"
    by auto
  have fin_index: "finite ({..m}::nat set)" by simp
  have fin_each: "!!i. i <= m ==> finite (?S i)"
    by (simp add: finite_bitstrings_of_len)
  have disj: "!!i j. i <= m ==> j <= m ==> i ~= j
    ==> ?S i Int ?S j = {}"
    by auto
```

```
  have "card (Union ((%i. ?S i) ` {..m})) = sum (%i. card (?S i)) {..m}"
    by (rule card_UN_disjoint) (use fin_index fin_each disj in auto)
  also have "... = sum (%i. 2 ^ i) {..m}"
    by (simp add: many_strings_of_length)
  finally show ?thesis
    by (simp add: union_eq)
qed
```

The pigeonhole argument

Theorem. For any injective encoding, there exists an incompressible bitstring.

```
theorem incompressible_strings_exist_for_enc:
  fixes enc :: "bitstring ⇒ bitstring"
  assumes "inj enc"
  shows "∃s. length s = m ∧ incompressible_by s enc"
proof (cases m)
  case 0
  have "length ([]::bitstring) = 0 ∧ 0 ≤ length (enc [])"
    by simp
  then show ?thesis using 0 incompressible_by_def compressible_def by
auto
next
  case (Suc r)
  let ?S = "{t::bitstring. length t = Suc r}"
  let ?T = "{u::bitstring. length u ≤ r}"
  have finS: "finite ?S" by (simp add: finite_bitstrings_of_len)
  have finT: "finite ?T" by (simp add: finite_bitstrings_le_len)
  have Sm: "card ?S = 2 ^ Suc r"
    by (simp add: many_strings_of_length)
  have Tm: "card ?T = sum (%i. (2::nat) ^ i) {..r}"
    by (simp add: card_bitstrings_le_len)
  also have "... = sum (%i. (2::nat) ^ i) {..<Suc r}"
    by (simp add: lessThan_Suc_atMost)
  finally have Tm': "card ?T = sum (%i. (2::nat) ^ i) {..<Suc r}" .
  from Tm' have lt: "card ?T = 2 ^ Suc r - 1"
    by (simp add: sum_pow2_lt)
  from Sm lt have card_less: "card ?T < card ?S" by simp
  have not_all_shrink: "~(ALL t:?S. length (enc t) <= r)"
  proof
    assume H: "ALL t:?S. length (enc t) <= r"
    have "enc ` ?S <= ?T" using H by auto
    hence "card (enc ` ?S) <= card ?T"
      using finT card_mono by blast
    moreover from assms finS have "card (enc ` ?S) = card ?S"
      using card_image by (metis subset_UNIV subset_inj_on)
    ultimately show False using card_less by linarith
  qed
```

```
  then obtain t where tS: "t : ?S" and len: "~ length (enc t) <= r"
    by blast
  hence "length (enc t) ≥ Suc r" by simp
  moreover from tS have "length t = Suc r" by auto
  ultimately show ?thesis
    using Suc incompressible_by_def compressible_def by auto
qed
```

Proof system locale

This locale states the information assumptions used in the argument. A bitstring p represents a proposed proof.

1. **Trace specification.** Let $L = \text{length}(p)$. If p proves that a particular positive n reaches 1, and
$$T^{(L)}(n) > 2,$$
then p contains an encoding of the parity vector
$$(n, T(n), \ldots, T^{(L)}(n)) \pmod 2.$$

2. **Universal instantiation.** If p proves the Collatz conjecture, then for every positive n it proves the instance
$$\exists k.\ T^{(k)}(n) = 1.$$

The parity encoding is assumed injective. The counting argument above then supplies a vector x of length $L + 1$ whose encoding has length at least $L + 1$.

Motivation for the trace-specification assumption

The trace-specification assumption is motivated by three structural properties of the Collatz map.

1. Realisability of all parity vectors For the Collatz map, every finite bitstring occurs as a parity vector. As a result, any proof of universal convergence must account for all possible parity patterns; none can be excluded a priori.

By contrast, consider the function
$$T_1(n) = \begin{cases} n/2 & \text{if } n \text{ is even,} \\ n+1 & \text{if } n \text{ is odd.} \end{cases}$$
This map is not realisable in the above sense: after an odd step, the next value is always even, so the parity pattern [True, True] never occurs. Proofs for T_1 therefore do not need to consider arbitrary parity strings.

2. Opposite monotonicity In the Collatz map, even steps decrease the value, while odd steps increase it. Consequently, descent of the value at every step cannot serve as a convergence argument.

By contrast, consider the function

$$T_2(n) = \begin{cases} n/2 & \text{if } n \text{ is even,} \\ (n+1)/2 & \text{if } n \text{ is odd.} \end{cases}$$

Both branches decrease for $n > 1$, so convergence to 1 follows by descent on the positive integers, without needing to specify the individual parity choices.

3. Injectivity of the affine formula For the Collatz map, we have the identity

$$T^k(n) = \frac{3^s \cdot n + c}{2^k},$$

where the parameters (k, s, c) are determined by the parity vector. As shown earlier, this correspondence is injective, so specifying the exact parameters also determines the full parity sequence.

Together, these properties motivate requiring proofs to encode the relevant parity information. This requirement is expressed by the trace-specification assumption below. The final result is conditional on that assumption: the Isabelle development does not derive it from the rules of an arbitrary formal proof system.

```
locale Collatz_Trace_Barrier =
  fixes enc_parity :: "bool list ⇒ bitstring"
    and proves_reaches_one :: "bitstring ⇒ nat ⇒ bool"
    and is_collatz_proof :: "bitstring ⇒ bool"
  assumes enc_parity_injective: "inj enc_parity"

  assumes trace_specification:
    "[| proves_reaches_one p n;
        n > 0;
        Tpow (length p) n > 2 |]
     ==> contains p
           (enc_parity (parity_vec n (Suc (length p))))"

  assumes collatz_proof_instances:
    "is_collatz_proof p ==> ALL n>0. proves_reaches_one p n"
begin

lemma incompressible_parity_encodings_exist:
  shows "∃s. length s = m ∧ incompressible_by s enc_parity"
  using incompressible_strings_exist_for_enc[OF enc_parity_injective]
.
```

Interpretation of the main theorem

Suppose that p is a proof and let $L = \text{length}(p)$. Choose an incompressible vector x of length $L+1$. The preceding realisability result supplies a positive n whose first $L+1$ parity values equal x and whose parities at steps L and $L+1$ are equal. Hence $T^{(L)}(n) > 2$, and if $T^{(k)}(n) = 1$, then $k > L$. The trace–specification assumption requires p to contain the encoded vector x. Its encoding has length at least $L+1$, whereas every substring of p has length at most L. This is the required contradiction.

```
theorem no_finite_collatz_proof:
  assumes p_proof: "is_collatz_proof p"
  shows False
proof -
  define L where "L = length p"
  obtain x where
    x_len: "length x = Suc L" and
    x_incomp: "incompressible_by x enc_parity"
    using incompressible_parity_encodings_exist[of "Suc L"]
    by blast
  obtain n where
    n_pos: "n > 0" and
    pv_eq: "parity_vec n (Suc L) = x" and
    same_parity:
      "odd (Tpow (Suc L) n) = odd (Tpow L n)"
    using parity_vector_realizable_with_matching_next_parity[OF x_len]
    by blast
  have value_at_L_gt_two:
    "Tpow (length p) n > 2"
    using equal_successive_parity_imp_gt_two[OF n_pos same_parity]
    by (simp add: L_def)
  have instance_proof: "proves_reaches_one p n"
    using collatz_proof_instances[OF p_proof] n_pos
    by blast
  have pv_eq':
    "parity_vec n (Suc (length p)) = x"
    using pv_eq by (simp add: L_def)
  have contains_x: "contains p (enc_parity x)"
  proof -
    have trace_contained:
      "contains p
        (enc_parity (parity_vec n (Suc (length p))))"
      using trace_specification[
        OF instance_proof n_pos value_at_L_gt_two]
      by (simp add: L_def)
    show ?thesis
      using trace_contained pv_eq' by simp
  qed
  have enc_large: "Suc L ≤ length (enc_parity x)"
```

```
    using x_incomp x_len
    by (simp add: incompressible_by_def compressible_def)
  have enc_fits: "length (enc_parity x) ≤ length p"
    using contains_len_bound[OF contains_x] .
  have "Suc L <= length p"
    using enc_large enc_fits by linarith
  then show False
    by (simp add: L_def)
qed

corollary no_collatz_proof_in_this_system:
  shows "¬ (∃p. is_collatz_proof p)"
  using no_finite_collatz_proof
  by blast

end
end
```